

Designing an Agentic Work System for Automated Exam Generation

2026-1학기 과학적관리 (박우진 교수님)

Homework #2

8조 강민서 서윤진 신병석 신지영 이상협 이체린 이호태

1. Introduction

1.1 Project Objective

시험 출제는 강의 내용을 분석하여 적절한 문제를 구성하고, 이에 대한 모범답안을 작성하는 과정을 포함한다. 그러나 이러한 작업은 많은 시간과 노력이 요구되며, 출제 범위의 균형, 문제 유형의 다양성, 난이도 조절 등 다양한 요소를 동시에 고려해야 한다.

본 프로젝트의 목표는 강의 노트와 교수자의 요구사항을 입력으로 받아 중간고사 시험지와 모범답안을 자동으로 생성하는 Agentic Work System을 설계하고 구현하는 것이다. 이를 통해 기존에 교수자나 조교가 수행하던 시험 출제 업무의 대부분을 자동화하고 효율적인 시험 생성 프로세스를 구축하고자 하였다. 또한 본 시스템은 단순히 문제를 생성하는 것에 그치지 않고, 실제 시험에 활용 가능한 형태의 시험지와 모범답안을 생성하는 것을 목표로 한다.

1.2 Key Idea

본 시스템의 핵심 아이디어는 단순한 문제 생성이 아닌 Human-in-the-Loop 기반의 Agentic Workflow를 구축하는 것이다. 실제 시험 출제 과정에서는 문제를 생성한 후에도 적절성, 난이도, 표현 방식 등을 검토하고 수정하는 과정이 필요하다. 따라서 모든 의사결정을 LLM에게 맡기기보다는 인간의 판단이 필요한 부분을 별도의 검수 단계로 분리하였다.

이를 위해 시스템은 세 단계의 공정으로 구성된다. 첫 번째 공정에서는 시험에 필요한 문제 수보다 많은 후보 문제를 생성하고 이를 검토하여 문제은행을 구축한다. 두 번째 공정에서는 생성된 문제은행 또는 최종 시험지 텍스트를 실제 활용 가능한 Word 문서 형태로 변환한다. 세 번째 공정에서는 인간 검수자가 선택한 문항과 수정 요청을 반영하여 최종 시험지와 모범답안을 구성한다. 이와 같은 구조를 통해 LLM의 생성 능력과 인간의 전문적인 판단을 결합하여 시험 출제의 효율성과 품질을 동시에 확보하고자 하였다.

2. System Architecture

2.1 Overall Architecture

아래 그림은 본 프로젝트에서 설계한 시험 자동 생성 시스템의 전체 구조를 나타낸다. 시스템은 강의 노트와 출제 요구사항을 입력으로 받아 시험 문제 생성, 품질 검증, 인간 검수, 시험지 재구성 및 최종 출력의 과정을 거쳐 최종 시험지와 모범답안을 생성한다.

먼저 사용자는 강의 노트와 함께 시험 범위, 문제 유형, 난이도 등 출제 요구사항을 입력한다. 시스템은 이를 기반으로 최종 시험에 필요한 문제 수보다 많은 후보 문제를 생성하여 문제은행을 구축한다. 이후 생성된 문제들은 범위 포함 여부, 문제 중복 여부, 난이도 균형, 문제 품질 등의 기준에 따라 자동 검증 과정을 거친다. 검증이 완료된 문제은행은 인간 검수자에게 제공된다. 검수자는 생성된 문제를 검토한 후 최종 시험에 포함할 문제를 선택하고, 필요한 경우 문제 수정 요청이나 추가 요구사항을 입력할 수 있다. 이러한 인간 검수 과정은 출제 의도 반영, 문제 품질 향상, 부적절한 문제 제거 등을 위해 수행된다.

마지막으로 시스템은 선택된 문제와 검수자의 피드백을 반영하여 시험지를 재구성하고 모범

답안을 생성한다. 생성된 결과물은 학생용 시험지와 정답 및 모범답안이 포함된 출제자용 문서로 분리하여 Word 파일 형태로 출력된다. 이를 통해 반복적이고 시간이 많이 소요되는 시험 출제 업무를 자동화하면서 최종 품질에 대한 인간의 통제를 유지할 수 있도록 설계하였다.

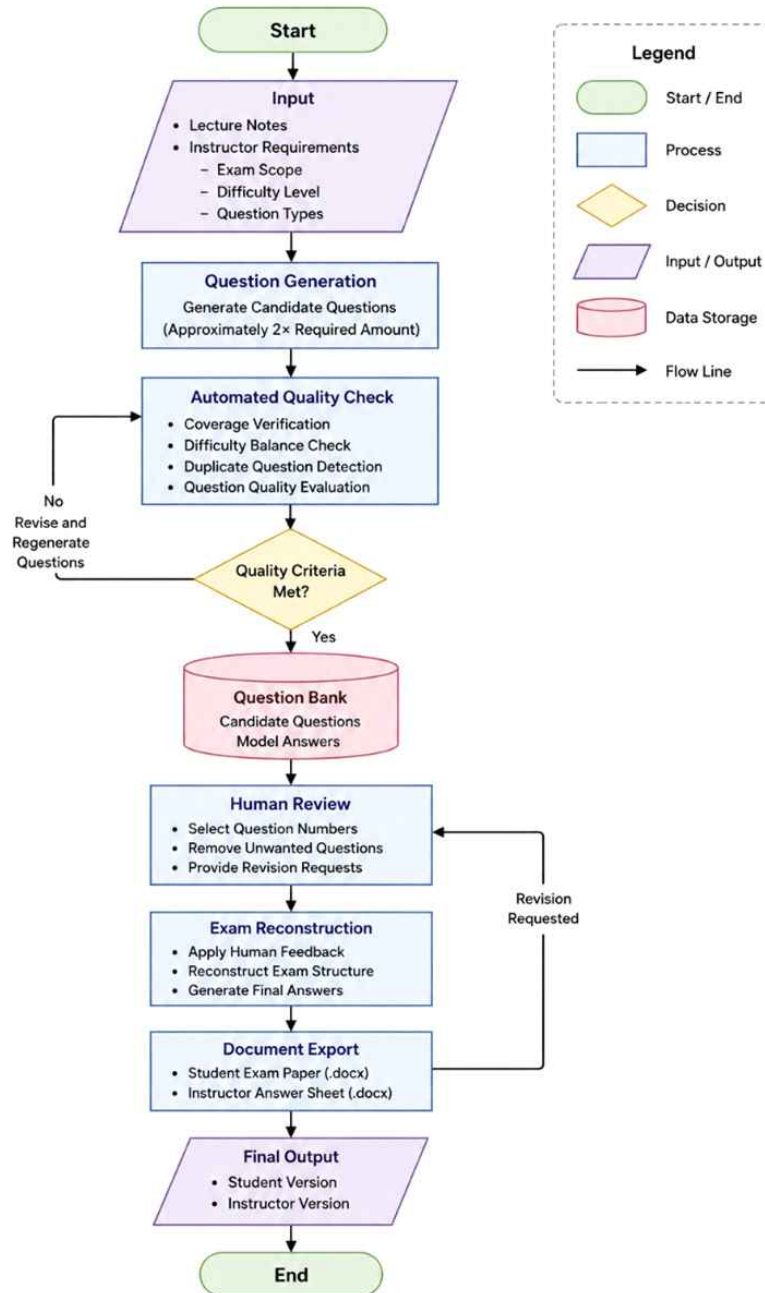


Figure 1. Overall Workflow of the Proposed Exam Generation System

아래 표는 시험 생성 과정에서 각 주체가 담당하는 업무와 해당 업무를 담당하는 이유를 정리한 것이다. 인간은 출제 의도와 평가 목표를 반영하는 의사결정 중심의 업무를 수행하며, 에이전트는 문제 생성, 품질 검토, 시험지 재구성 및 문서 생성과 같은 반복적이고 자동화 가능한 업무를 담당한다.

Workflow Stage	Responsibility	Why Human Involvement Is Required
Lecture Note Input	강의노트 제공	시험 범위로 인정할 자료 선별
Instructor Requirement Input	시험 범위, 난이도, 문제 유형 입력	출제 의도와 평가 목표 반영
Question Selection	최종 시험에 포함할 문제 선택	자동 생성 결과 중 적절한 문제 선별
Revision Request Input	수정사항 및 추가 요구사항 입력	교수자의 전문적 판단 반영

Table 1. Human Responsibilities in the Proposed System

Workflow Stage	Responsibility	Why AI Agent Is Suitable
Question Generation	후보 문제 생성	대량의 문제를 빠르게 생성
Quality Review	문제 품질 검토	중복, 범위 편중, 난이도 불균형 자동 검출
Question Bank Creation	문제은행 구성	생성 결과를 체계적으로 정리
Exam Reconstruction	최종 시험지 재구성	인간 피드백을 신속하게 반영
Model Answer Generation	모범답안 생성	문제별 답안을 자동 작성
Document Export	Word 문서 생성	서식 적용 및 파일 생성 자동화

Table 2. AI Agent Responsibilities in the Proposed System

2.2 Agent Descriptions

본 시스템은 시험 생성 업무를 하나의 단일 작업으로 처리하지 않고, 실제 코드 구현 구조에 따라 세 개의 주요 공정으로 분리하였다. 각 공정은 입력 자료 처리, 문제 생성 및 검토, 최종 시험지 구성, 문서 출력이라는 서로 다른 역할을 담당한다. 이러한 분리는 각 기능의 책임을 명확히 하고, 생성 결과를 단계별로 확인 및 수정할 수 있도록 하기 위한 것이다.

2.2.1 Process 1: Candidate Question Bank Generation and Review

첫 번째 공정은 강의자료와 출제 요구사항을 기반으로 후보 문제를 생성하고 이를 검토하여 문제은행을 구축하는 단계이다. 시스템은 시험 범위와 난이도, 문제 유형 등의 요구사항을 반영하여 최종 시험에 필요한 문항 수보다 많은 후보 문제를 생성한다. 이후 생성된 문제들에 대해 범위 포함 여부, 문제 중복 여부, 난이도 적절성 및 전반적인 문제 품질을 검토하여 수정된 문제은행을 구성한다.

이 공정의 목적은 다양한 후보 문제를 확보하고, 이후 단계에서 적절한 문제를 선택할 수 있

도록 충분한 선택지를 제공하는 데 있다.

2.2.2 Process 2: Exam Document Export

두 번째 공정은 생성된 문제은행 또는 시험지 텍스트를 실제 활용 가능한 Word 문서 형태로 변환하는 단계이다. 시스템은 생성된 결과물을 문서 형식에 맞게 정리하고, 제목, 표, 문단 구조 등의 서식을 자동으로 구성한다. 또한 필요에 따라 학생용 시험지와 교수용 답안지를 분리하여 출력할 수 있도록 설계하였다.

이 공정의 목적은 LLM이 생성한 텍스트 결과물을 실제 제출 및 활용이 가능한 문서 형태로 변환하는 데 있다. 특히 본 공정은 특정 단계에 종속되지 않는 독립적인 출력 모듈로 설계되어 문제은행 출력과 최종 시험지 출력 모두에 재사용된다. 이를 통해 문서 생성 기능의 중복 구현을 방지하고 시스템의 유지보수성을 향상시킬 수 있다.

2.2.3 Process 3: Feedback-Based Exam Construction

세 번째 공정은 인간 검수자가 선택한 문항과 수정 요청을 반영하여 최종 시험지와 모범답안을 구성하는 단계이다. 사용자는 문제은행을 검토한 후 최종 시험에 포함할 문제를 선택하고, 필요한 경우 문제 수정, 배점 조정, 문제 유형 변경 등의 추가 요구사항을 입력할 수 있다. 시스템은 이러한 선택 결과와 수정 요청을 반영하여 문제를 재구성하고 최종 시험지와 모범답안을 생성한다. 이를 통해 자동 생성의 효율성과 인간의 전문적인 판단을 결합하여 실제 시험에 활용 가능한 결과물을 제공한다.

3. Implementation

3.1 Development Environment

본 시스템은 Python 기반으로 구현되었으며, Google Colab 환경에서 실행하였다. 또한 Gemini API를 활용하여 문제 생성, 문제 검토, 그리고 최종 시험지 구성을 수행하였다. 시스템은 문제은행 생성 및 검토, 문서 출력, 최종 시험지 구성의 세 공정으로 구성된다.

3.2 Implementation of Each Process

3.2.1 Process 1: Candidate Question Bank Generation and Review

Process 1은 일차적으로 시험지에 활용할 수 있는 후보 문항들로서 문제은행을 생성하는 단계로, 문제은행은 필요 문항 수의 2배수를 출제하도록 설계하였다. 강의자료 수집, 후보 문제 생성, 문제 검토의 세 단계로 구현하였다. 각 기능을 독립적인 함수로 분리함으로써 유지보수성과 재사용성을 높이고, 문제 생성 과정의 각 단계를 개별적으로 검증할 수 있도록 설계하였다.

Step 1. Lecture Note Integration

첫 번째 단계에서는 여러 개의 PDF 및 TXT 형식의 강의자료를 하나의 통합 텍스트로 병합한다. 강의자료를 사전에 통합함으로써 이후 문제 생성 과정에서 전체 강의 범위를 일관되게 참조할 수 있도록 하였다. 이를 통해 특정 강의자료에만 편중된 문제가 생성되는 것을 방지하고, 시험 범위 전체를 반영한 문제 생성을 가능하게 하였다.

```

## dir_path에 있는 모든 파일을 읽어서 하나로 합침
def load_all_lecture_notes_from_dir(dir_path):
    import os, glob
    from pypdf import PdfReader
    pdf_files = glob.glob(os.path.join(dir_path, "*.pdf"))
    txt_files = glob.glob(os.path.join(dir_path, "*.txt"))
    all_files = sorted(pdf_files + txt_files)
    total_files = len(all_files)
    if not all_files: return ""
    print(f"총 {total_files}개 파일 로딩 시작...")
    merged_context = ""
    for idx, path in enumerate(all_files, start=1):
        file_name = os.path.basename(path)
        print(f" [{idx}/{total_files}] 로드 중: {file_name}")
        if path.lower().endswith(".pdf"):
            try:
                reader = PdfReader(path)
                for page in reader.pages:
                    text = page.extract_text()
                    if text: merged_context += text + "\n"
            except: pass
        else:
            try:
                with open(path, 'r', encoding='utf-8') as f:
                    merged_context += f.read() + "\n"
            except: pass
    return merged_context

```

Step 2. Candidate Question Generation

두 번째 단계에서는 통합된 강의자료와 사용자가 입력한 출제 요구사항을 기반으로 후보 문제를 생성한다. 이때 최종 시험에 필요한 문항 수로 입력된 문항 수(약술형:short_count, 논술형:essay_count)보다 2배수에 해당하는 문제를 생성하도록 설계하였다. 이는 인간 검수자가 다양한 문제 중에서 적절한 문제를 선택할 수 있도록 충분한 선택지를 제공하기 위함이며, 최종 시험지의 품질 향상에 목적이 있다.

또한 문제은행 생성 과정에서 출제 가이드라인을 함께 입력하여 생성되는 문항의 품질을 향상시키고자 하였다. 구체적으로는 단순 암기형 문제보다는 핵심 개념의 본질적인 이해를 평가할 수 있도록 하였으며, 학습자의 사고력과 개념 적용 능력을 확인할 수 있는 문항을 우선적으로 생성하도록 지시하였다. 또한 계산형 문제의 경우 과도하게 복잡한 연산을 요구하기보다는 강의에서 학습한 개념과 공식을 적절히 활용할 수 있는 수준의 연산 복잡도를 유지하도록 설계하였다. 이를 통해 문제은행 초안이 실제 평가 목적에 부합하는 양질의 문항들로 구성될 수 있도록 하였다.

```

def agent_advanced_bank_generator(lecture_note, short_count, essay_count, detailed_requirements):
    """[Phase 1-A] 단답형 및 주관식 2배수 문제은행 초안 생성"""
    bank_short = short_count * 2
    bank_essay = essay_count * 2

    print(f" [Bank Generator] 1차 2배수 초안 풀(Pool) 구성 중...")

    system_instruction = f"""당신은 과학적관리 과목의 수석 출제위원입니다. 제공된 강의 데이터 소스를 바탕으로 아래 요구하는 수량의 문제은행 초안을 작성하십시오.

    [출제 수량 제약조건]:
    - 약술형 문제: 정확히 {bank_short}문제를 출제할 것. (핵심 개념 용어나 정의, 계산 결과를 간결하게 묻고 답하는 형태, 독립적으로 1번부터 {bank_short}번 부여)
    - 주관식(서술/논술/계산형) 문제: 정확히 {bank_essay}문제를 출제할 것. (심도 있는 메커니즘 분석 및 계산 수식 활용 형태, 약술형에 이어서 {bank_short+1}번부터 {bank_essay}문제에 번호 부여)

    [인간 검수자의 상세 지시]: {detailed_requirements}

    반드시 다음 [좋은 문항 출제 기준]에 맞추어 문항을 구성해야 합니다.
    [좋은 문제의 출제 기준]:
    1. 개념의 본질 지향: 단순 지식의 파편만을 묻지 말고, 그 이론이나 메커니즘이 왜 등장했는지 본질과 배경에 대한 이해를 묻는 문제로 설계할 것.
    2. 사고력 및 이해도 평가: 단순한 암기식 문항은 배제하고, 개인의 생각이나 비판적 시각을 요구하는 문항으로 설계할 것.
    3. 적정 연산 복잡도: 손으로 계산할 수 있는 수준의 문항으로서, 필요 이상으로 계산이 꼬여서 핵심 개념 파악을 방해하는 문항은 지양할 것.

    각 문항은 반드시 [문제 내용], [출제의도], [난이도 및 단위 정보], [모범 답안 및 상세 풀이]를 포함해야 합니다."""

    prompt = f"""위 제약조건에 맞춰 최고 품질의 전문적인 문제은행 초안을 빌드하세요.\n\n[강의 노트]:\n{lecture_note}"
    response = client.models.generate_content(
        model='gemini-2.5-flash', contents=prompt, config=types.GenerateContentConfig(system_instruction=system_instruction)
    )
    return response.text

```

Step 3. Question Review

세 번째 단계에서는 생성된 후보 문제를 자동으로 검토한다. 문제 생성과 검토 기능을 분리함으로써 생성 결과를 독립적으로 평가할 수 있도록 하였으며, 문제 중복, 범위 편중, 난이도 불균형 등의 문제를 최소화하고자 하였다. 또한 문제 표현의 적절성과 출제 요구사항 충족 여부를 함께 검토하여 문제은행의 품질을 향상시키고자 했다.

곧, 이와 같은 일련의 과정은 의도적으로 문항의 수를 확장하여 다양성을 높이고, 각 문항에 대한 가이드라인을 설정하며, 출제-검토 단계를 분리하여 수행하도록 함으로써 일차로 생성된 문항들이 양적, 질적 기준 모두를 만족할 수 있도록 설계한 것이다.

+ 검토 과정은 Gemini 2.5 Pro 모델을 사용: 검토 과정을 Flash 모델에 비해 더 고도화할 수 있도록 의도적으로 선택하였음을 명시.

```
def agent_bank_reviewer(raw_bank, lecture_note, detailed_requirements):
    """[Phase 1-B] 성찰 패턴(Reflection) 기반 정정 검토 에이전트"""
    print("🔍 [Bank Reviewer] 생성된 문항 자율 검토 및 품질 튜닝 중...")
    system_instruction = (
        "당신은 과학교과리 과목의 '수석 문항 검토위원'입니다. 전 단계에서 생성된 2배수 문제은행 초안을 1차로 심사하십시오.\n\n"
        "📌 [심사 및 정정 가이드라인]:\n"
        "1. 발문 심사: 문장이나 발문이 불필요하게 비대하거나 증언부연하는 경우 핵심 지식 위주로 정제하고 문장력을 명확하게 교정하십시오.\n"
        "2. 무결성 검증: 계산 문제의 수식과 수치 데이터가 강의노트 범위와 완벽히 일치하는지 검토하고, 답이 명확하게 떨어지는지 면밀히 검토하여 할루시네이션을 바로잡으십시오.\n"
        "3. 포맷 명확화: 인간 검수자가 최종 선별할 번호를 고르기 가장 편하도록 약술형 파트와 주관식 파트를 잘 구분하십시오.\n"
        "4. 문항의 질 : [좋은 문제의 출제 기준] 3가지에 비추어, 문항의 수준을 검토하여 문항을 1차로 고도화하여 수정하십시오.\n\n"
        "오직 검토와 교정이 끝난 '완벽한 품질의 2배수 문제은행 원안지 통합본'만 반환하십시오."
    )
    prompt = f"[인간 상세 지시사항]:\n{detailed_requirements}\n\n[1차 초안 문제은행]:\n{raw_bank}\n\n[검증용 강의노트]:\n{lecture_note}"
    response = client.models.generate_content(
        model='gemini-2.5-pro', contents=prompt, config=types.GenerateContentConfig(system_instruction=system_instruction)
    )
    return response.text
```

Step 4. Question Bank Document Generation

마지막으로, 4번째 단계에서는 생성된 문제은행을 Word 문서 형태로 저장한다. 이는 ‘출력기’의 첫 번째 파트로 구성되었으며, docx 라이브러리를 import하여 바로 Word 문서의 형식으로 출력되도록 하였다. 곧, 문제은행을 별도의 문서로 출력함으로써 인간 검수자가 생성 결과를 쉽게 확인할 수 있도록 하였으며, 이후 최종 시험지 구성 단계에서 사용할 문제를 선택할 수 있도록 하였다.

```

## 문제은행 (전체 문항) 출력기
def build_bank_docx(bank_text, filename):
    import os, re
    from docx import Document
    from docx.shared import Pt, Inches

    doc = Document()
    for s in doc.sections: s.top_margin = s.bottom_margin = s.left_margin = s.right_margin = Inches(1.0)
    style = doc.styles['Normal']
    style.font.name, style.font.size = 'Arial', Pt(11)
    style.paragraph_format.line_spacing = 1.25
    style.paragraph_format.space_after = Pt(8)

    doc.add_heading("중간고사 2배수 확장 문제은행 원안지", 0)
    lines = bank_text.split('\n')
    in_table = False
    table_data = []

    for line in lines:
        ls = line.strip()
        if not ls:
            if in_table and table_data:
                render_word_table(doc, table_data)
                table_data = []
                in_table = False
            continue

        if ls.startswith('|'):
            in_table = True
            if re.search(r'^[|\s:-]+$', ls): continue
            cells = [c.strip().replace("##", "") for c in ls.split('|')[1:-1]]
            table_data.append(cells)
            continue
        else:
            if in_table and table_data:
                render_word_table(doc, table_data)
                table_data = []

```

```

        line_cleaned = ls.replace("##", "").replace("_", "").strip()
        p = doc.add_paragraph()
        if ls.startswith('#'):
            run = p.add_run(line_cleaned.replace('#', '').strip())
            run.font.bold, run.font.size = True, Pt(14)
        elif any(line_cleaned.startswith(x) for x in ["*", "[", "정답", "출제의도", "문제", "문항", "난이도"]):
            p.add_run(line_cleaned).font.bold = True
        elif len(line_cleaned) > 0 and line_cleaned[0].isdigit() and ("," in line_cleaned or "번" in line_cleaned or ")" in line_cleaned):
            p.add_run(line_cleaned).font.bold = True
        else:
            p.add_run(line_cleaned)

    if in_table and table_data: render_word_table(doc, table_data)
    doc.save(filename)
    print(f"📄 문제은행 원안 워드 출력 완료 -> '{filename}')
```

3.2.2 Process 2: Feedback-Based Exam Construction

Process 2는 검토 완료된 문제은행을 바탕으로 인간 검수자의 선택과 편집 지시를 반영하여 최종 시험지와 모범답안을 생성하는 공정이다. 이 공정은 Human Feedback Collection, Final Exam Construction, Document Export의 세 단계로 구성된다. agent_asymmetric_exam_builder() 함수는 코드상 앞부분에 정의되어 있지만, 기능적으로는 인간 검수자의 문항 선택과 편집 지시를 반영하여 최종 시험지를 구성하는 역할을 하므로 Process 3에 포함하였다.

Step 1. Human Feedback Collection

첫 번째 단계에서는 인간 검수자가 검토 완료된 문제은행을 확인한 후 최종 시험에 사용할 문항 번호와 추가 편집 지시사항을 입력할 수 있는 기회를 만들었다. 문항 번호 선택은 자동 생성된 후보 문제 중 실제 시험에 적합한 문제를 인위적으로 선별하기 위한 과정이며, 편집 지시사항은 OX 형식 제외, 중복 문제 대체, 배점 조정, 문제 유형 제한 등 출제자의 의도를

반영하기 위한 정보로 사용된다. 이를 통해 완전 자동화 과정에서 발생할 수 있는 품질 문제를 줄이고, Human-in-the-loop 방식으로 최종 시험지의 신뢰성을 높일 수 있도록 하였다.

```
## 최종 시험지 및 모범답안 출력

#
# [인간] 검수자의 지시사항 입력 (OX 원천 차단 레이아웃 반영)
#

# 1. 사용할 문항 번호를 명시하세요.
selected_numbers = ""
- 선택한 약술형 후보군: 1, 2, 4, 7, 8
- 선택한 주관식 후보군: 11, 12, 13, 15, 16, 17, 20
""

# 2. OX 원형 출처 링크 제거 + 편집 지시사항 입력 + 배점 자동 입력. 특정 문항을 정해 수정 방향을 제시하는 것도 가능함.
specific_edits = (
    "[출제 유형 변경 제한]: 최종 시험지의 약술형 문항들은 절대로 'O/X'를 선택하는 문제 '문' 만들지 마십시오. ",
    "오직 특정 용어, 핵심 개념 단어, 혹은 한 줄 이내의 핵심 문장을 직접 원문으로 작성해야 하는 '순수 주관식 약술형'으로만 구성해야 합니다. ",
    "만약 원천지에 OX 형태로 가공된 문제가 있다면, 이를 즉시 '무엇이라 하든지 쓰시오' 형태의 순수 약술형 문항으로 문장 구조를 리팩토링하여 반영하십시오.\n\n",
    "- 추가 지시: 선택한 문항 중 주제가 완벽하게 겹치는 문제가 발견되면 원안지 풀(Pool) 내의 다른 번호로 자를 대체해도 좋습니다. ",
    "단답형과 주관식 수량 비율을 정확히 유지하고, 전체 문항은 약술형 40점, 주관식 60점으로 총합 100점 만점이 되도록 난이도별 배점을 정확하게 밸런싱하십시오."
)
)
```

Step 2. Final Exam Construction

두 번째 단계에서는 선택된 문항 번호와 편집 지시사항을 agent_asymmetric_exam_builder() 에 전달하여 최종 시험지와 모범답안 초안을 생성한다. agent_asymmetric_exam_builder() 함수는 인간 검수자가 선택한 문항 번호(selected_numbers)와 편집 지시사항(specific_edits), 검토 완료된 문제은행(final_reviewed_bank), 그리고 원본 강의자료(merged_notes)를 입력으로 받아 최종 시험지와 모범답안을 생성한다.

```
def agent_asymmetric_exam_builder(selected_cfg, edit_instructions, reviewed_bank, lecture_note):
    """[Phase 2] 인간 선택 반영 및 유연한 대체 선택 알고리즘이 내장된 최종 리파이너"""
    print("🔴 [Asymmetric Builder] 인간 선택 문항 수정, 주제 중복 자동 필터링 및 100점 만점 최적화 중...")

    system_instruction = (
        "당신은 최종 시험지 렌더링 자를 에이전트입니다. 검토 완료된 문제은행 원안에서 인간 검수자가 확정한 선택 번호를 기반으로 최종 시험지를 구축하십시오.\n\n"
        "🔴 [대체 선택 유연성 규칙]:\n"
        "- 인간 검수자가 특정 문제 번호들을 지정해 주었으나, 만약 선택된 문제들 간에 '주제나 핵심 개념이 심각하게 중복'된다고 판단되는 경우, "
        "지정된 번호 외에 문제은행 원안지 풀에 포함된 다른 고품질 예비 문항을 자율적으로 대체 선택하여 출제할 수 있는 권한을 부여합니다. 단, 최종 출제 수량은 엄격히 유지하십시오.\n\n"
        "🔴 [최종 시험지 구성 지침]:\n"
        "1. [약술형 배치]: 선택된 단답형 문제들은 지면을 아끼기 위해 대문항 하나 아래에 배치하거나 전반부에 조밀하게 배치하십시오.\n"
        "2. [주관식 배치]: 서술형 및 계산형 주관식 문항들은 한 페이지에 한 문제씩 여유롭게 들어갈 수 있도록 각각 명확한 독립 대문항 번호(예: 문항 2, 문항 3...)를 부여하십시오.\n"
        "3. [100점 만점 스케일링]: 최종 완성된 모든 대문항들의 배점 합계가 정확하게 총합 '100점'이 되도록 난이도를 고려하여 공정하게 분배하십시오.\n"
        "4. ⚠️ 중요: 출력을 본문 내에 반드시 명확하게 '-----학생용 문제지-----' 문자열 표시와 '-----교수용 해설지-----' 문자열 표시를 넣어 파트 구분을 완벽히 격리하십시오."
    )

    prompt = (
        f"[인간 검수자 최종 선택 및 대체 권한 정보]:\n{selected_cfg}\n\n"
        f"[추가 수정 및 편집 요청]:\n{edit_instructions}\n\n"
        f"[데이터 원천 - 검토 완료된 2배수 문제은행 원안지]:\n{reviewed_bank}\n\n"
        f"[강의자료 참조]:\n{lecture_note}"
    )

    response = client.models.generate_content(
        model='gemini-2.5-pro', contents=prompt, config=types.GenerateContentConfig(system_instruction=system_instruction)
    )
    return response.text
```

생성 과정에서는 선택된 문항 간의 중복 여부를 검토하고 필요 시 다른 후보 문항으로 대체하며, 문제 유형 구성과 배점 체계를 조정하여 최종 시험지 텍스트(final_exam_text)를 생성한다.

```
# -----
# 📌 후공정 최종 리팩토링 및 감제 파일 토크 세션 가동
# -----

final_exam_text = agent_asymmetric_exam_builder(
    selected_cfg=selected_numbers,
    edit_instructions=specific_edits,
    reviewed_bank=final_reviewed_bank,
    lecture_note=merged_notes
)
```

3.2.3 Process 3: Exam Document Export

Process 3는 LLM이 생성한 텍스트 결과물에 대하여, 실제 활용 가능한 Word 문서 형태의 시험지로 변환하는 공정이다. 해당 공정은 앞선 문제은행 문서 생성 과정 및 학생용/교수용 문서 분리, 문서 서식 적용 및 저장 기능으로 구성된다. 본 공정은 독립적인 출력 모듈로 설계되었는데, 특히 이 단계에서는 최종 시험지를 출력하는 단계를 중심으로 살펴보고자 한다.

Step 1. Student and Instructor Version Separation

첫 번째 단계에서는 생성된 시험지 텍스트를 학생용 문제지와 교수용 답안지로 분리한다. 학생용 문서에는 시험 문제만 포함되며, 교수용 문서에는 문제와 함께 모범답안 및 채점 기준이 포함된다. 이를 통해 동일한 생성 결과를 사용자 목적에 맞게 활용할 수 있도록 한 것으로 하나의 파일에 모든 내용이 포함된 것과 비교하였을 때 활용성이 높다.

```
## 실제 최종 결과물 출력 엔진 (학생용 문제지 + 답안지)
def build_split_docx(final_text, filename_prefix):
    import os, re
    from docx import Document
    from docx.shared import Pt, Inches

    q_markers = ["---학생용 문제지---", "학생용 문제지", "---학생용 문제---", "[학생용 문제지]"]
    a_markers = ["---교수용 해설지---", "교수용 해설지", "---교수용 해설---", "[교수용 해설지]", "모범 답안", "정답 및 해설"]

    q_index = -1
    for m in q_markers:
        if m in final_text:
            q_index = final_text.find(m) + len(m)
            break

    a_index = -1
    for m in a_markers:
        if m in final_text:
            a_index = final_text.find(m)
            break

    if q_index == -1 or a_index == -1:
        half = len(final_text) // 2
        q_text = final_text[:half]
        a_text = final_text[half:]
    else:
        if q_index < a_index:
            q_text = final_text[q_index:a_index]
            a_text = final_text[a_index + 1:]
        else:
            a_text = final_text[a_index:q_index]
            q_text = final_text[q_index + 1:]
```

Step 3. Document Formatting and Export

마지막 단계에서는 Word 문서의 제목, 문단, 표, 페이지 구분 등의 서식을 자동으로 적용하고 최종 파일을 생성한다. 또한 LLM이 생성한 Markdown 형식의 표를 Word 표 형식으로 변환하여 문서의 가독성을 향상시켰다. 이를 통해 생성된 결과물을 별도의 편집 없이 바로 활용할 수 있도록 하였다. 학생용 시험지를 생성하는 과정에서는 각 문항 단위로 강제로 페이지를

바꾸도록 코드를 구성함으로써 시험지의 가독성을 높이고 편집에의 완결성을 높였다.

```
def save_docx(text, title, filename, is_question_paper):
    doc = Document()
    for s in doc.sections: s.top_margin = s.bottom_margin = s.left_margin = s.right_margin = Inches(1.0)
    style = doc.styles['Normal']
    style.font.name, style.font.size = 'Arial', Pt(11)
    style.paragraph_format.line_spacing = 1.25
    style.paragraph_format.space_after = Pt(8)

    doc.add_heading(title, 0)
    is_first = True
    lines = text.split('\n')
    in_table = False
    table_data = []

    for line in lines: ## 줄바꿈 처리기
        ls = line.strip()

        if any(tag in ls.lower() for tag in ["<br>", "<br/>", "<br>", "<hr>"]):
            continue
        if "다음 페이지에 계속" in ls:
            continue
        if ls == "---" or ls == "===":
            continue

        # [신규 필터 레이아웃] 문항 번호 옆이나 본문에 박힌 '[주관식 - 서술]', '[단답형]' 등의 대괄호 태그 제거
        # 배점 정보(예: [15점], [10점])는 남겨두기 위해 텍스트가 포함된 유형 성격의 대괄호만 정밀 타격합니다.
        ls = re.sub(r'[(주관식|단답형|서술형|계산형|논술형|객관식|OX형)](?:[^\s]*)', '', ls).strip()

        if not ls:
            if in_table and table_data:
                render_word_table(doc, table_data)
                table_data = []
                in_table = False
            continue
```

```
        if ls.startswith('('):
            in_table = True
            if re.search(r'[(\s:-]*$', ls): continue
            cells = [c.strip().replace("**", "") for c in ls.split('(')[1:-1]]
            table_data.append(cells)
            continue
        else:
            if in_table and table_data:
                render_word_table(doc, table_data)
                table_data = []
                in_table = False

            line_cleaned = ls.replace("**", "").replace("_", "").strip()
            clean_ls = re.sub(r'[\s\S-]', '', line_cleaned)
            is_new_macro_q = clean_ls.startswith("문항") or clean_ls.startswith("문제")

            if is_question_paper and is_new_macro_q:
                if not is_first: doc.add_page_break()
                is_first = False

            p = doc.add_paragraph()
            if ls.startswith('#'):
                run = p.add_run(line_cleaned.replace('#', '').strip())
                run.font.bold, run.font.size = True, Pt(14)
            elif is_new_macro_q or any(line_cleaned.startswith(x) for x in ["*", "[", "배점", "정답", "출제의도"]):
                p.add_run(line_cleaned).font.bold = True
            elif len(clean_ls) > 0 and clean_ls[0].isdigit() and ("." in line_cleaned or "번" in line_cleaned or ")" in line_cleaned):
                p.add_run(line_cleaned).font.bold = True
            else:
                p.add_run(line_cleaned)

        if in_table and table_data: render_word_table(doc, table_data)
        doc.save(filename)
        print(f"📄 독립 산출물 파일 동기화 완료 -> '{filename}'")

    save_docx(q_text, "2026학년도 제1학기 [과학적관리] 중간고사 문제지", f"{filename_prefix}_Questions.docx", True)
    save_docx(a_text, "2026학년도 제1학기 [과학적관리] 모범 답안 및 채점 해설서", f"{filename_prefix}_Answers.docx", False)
```

```

## 표 출력을 위한 데이터
def render_word_table(doc, table_data):
    if not table_data: return
    from docx.shared import Pt
    rows = len(table_data)
    cols = max(len(r) for r in table_data)
    table = doc.add_table(rows=rows, cols=cols)
    table.style = 'Light Shading Accent 1'
    for r_idx, row in enumerate(table_data):
        for c_idx, cell_value in enumerate(row):
            if c_idx < len(row):
                cell = table.cell(r_idx, c_idx)
                cell.text = cell_value
                if r_idx == 0:
                    for paragraph in cell.paragraphs:
                        for run in paragraph.runs: run.font.bold = True

```

Step 3. Document Export

마지막 단계에서는 Step 2에서 생성된 최종 시험지 텍스트(final_exam_text)를 Word 문서 형태로 변환하여 저장한다. 이 단계는 build_split_docx() 함수를 이용하여 수행된다. build_split_docx() 함수는 하나의 생성 결과물에 포함된 학생용 문제지와 교수용 모범답안을 자동으로 분리하고, 각각 독립된 Word 문서로 저장한다. 또한 문항 번호, 제목, 표, 페이지 구분 등의 서식을 적용하여 문서의 가독성을 향상시키며, LLM이 생성한 Markdown 형식의 표를 Word 표 형식으로 변환한다. 이를 통해 생성된 결과물을 추가적인 수작업 편집 없이 바로 활용할 수 있도록 하였다. 생성된 결과물은 학생용 시험지(Questions.docx)와 교수용 모범답안(Answers.docx) 형태로 저장되며, Process 2에서 구현한 문서 출력 모듈을 재사용함으로써 코드 중복을 줄이고 시스템의 유지보수성과 확장성을 향상시켰다.

```

print("\n📄 최종 정제 완료. 워드 파일 생성을 시작합니다...")
build_split_docx(final_exam_text, "HW2_Scientific_Management_Final")

```

4. Generated Output

4.1 Generated Midterm Exam

본 시스템을 활용하여 [과학적 관리] 과목의 중간기말고사 시험지를 실제로 생성하였다. 생성된 시험지는 강의 노트와 출제 요구사항을 기반으로 자동 생성되었으며, 실제 중간고사 시험지와 같이 약술형 및 논술형 문제가 시험지 범위 전체에 걸쳐 고루 구성될 수 있도록 함을 목표로 하였다.

생성된 시험지는 총 8개 문항으로 구성되며, 약술형, 계산형, 응용형, 논술형 문제를 포함하고 있다. 약술형 문항에서는 디지털 트윈, 서블릭(Therbligs), 사이클 시간(Cycle Time), 흐름 시간(Flow Time), 작업자-기계 시스템 등 핵심 개념에 대한 이해를 평가하도록 설계하였다. 또한 라인 밸런싱, 시간 연구(Time Study), 작업자-기계 시스템 분석과 같은 계산 문제를 포함하여 학생들의 정량적 문제 해결 능력을 평가할 수 있도록 구성하였다. 뿐만 아니라 병목(Bottleneck), 공정 수준 움직임 연구(Process-level Motion Study), 디지털 트윈과 인간 복지의 관계 등을 주제로 한 응용 및 논술형 문항을 포함하여 단순 암기가 아닌 개념의 적

용 능력과 비판적 사고 능력을 평가할 수 있도록 하였다.

생성된 시험지는 강의 범위를 전반적으로 포괄하면서도 문제 유형의 다양성을 확보하였으며, 계산 문제와 서술형 문제를 적절히 혼합하여 난이도 균형을 유지하였다. 또한 단순 정의 암기에 집중되지 않고 실제 산업 현장 적용, 시스템적 사고, 비판적 분석 능력을 요구하는 문항을 포함함으로써 실제 중간고사와 유사한 수준의 평가 도구를 생성할 수 있음을 확인하였다.

4.2 Generated Model Answers

생성된 시험지와 함께 각 문항에 대한 모범답안 및 채점 해설도 자동 생성하였다. 생성된 답안은 단순 정답 제시가 아닌 채점 기준으로 활용 가능한 수준의 상세한 해설을 포함하도록 설계하였다.

개념형 문제의 경우 핵심 정의와 주요 특징, 예시를 체계적으로 제시하였으며, 계산형 문제의 경우 공식 적용 과정과 계산 절차를 단계적으로 설명하였다. 예를 들어 라인 밸런싱 문제에서는 목표 사이클 시간 계산, 최소 작업 스테이션 수 산출, Largest Candidate Rule 적용 과정, 효율 및 지연 계산까지 모두 포함하였다. 또한 시간 연구 및 작업자-기계 시스템 문제에서도 계산 과정과 결과 해석을 함께 제시하였다. 논술형 문제에서는 단순한 정답 나열이 아니라 논리적 근거와 비판적 관점을 포함한 예시 답안을 생성하였다. 특히 디지털 트윈과 과학적 관리의 관계, 인간 복지의 중요성 등에 대한 답안은 강의 내용의 핵심 개념을 기반으로 하면서도 현대적 시각에서의 해석을 포함하고 있다.

이를 통해 본 시스템은 시험 문제 생성뿐만 아니라 모범답안 및 채점 기준 작성 업무까지 자동화할 수 있음을 확인하였다. 결과적으로 제안한 Agentic Work System은 시험 출제 과정의 상당 부분을 자동화하면서도 실제 교육 현장에서 활용 가능한 수준의 결과물을 생성할 수 있음을 보여주었다.

5. Discussion

5.1 Identified Limitations

(1) Technical Limitations

본 시스템은 Gemini API 기반의 LLM을 활용하여 시험 문제와 모범답안을 생성한다. 그러나 LLM 특성상 Hallucination 문제가 발생할 수 있으며, 강의자료에 존재하지 않는 개념이나 부정확한 설명이 포함될 가능성이 존재한다. 또한 동일한 입력에 대해서도 실행 시마다 다른 결과가 생성될 수 있기 때문에 문제의 난이도와 품질이 항상 일정하게 유지된다고 보장하기 어렵다. 특히 논술형 문제의 경우 답안의 길이와 깊이가 실행마다 달라질 수 있으며, 일부 문제는 지나치게 쉬워지거나 반대로 과도하게 어려워질 수 있다. 추가적으로 시스템은 문제의 교육적 적절성을 완전히 판단할 수 없다. 따라서 출제 의도와 학습 목표에 부합하는지 확인하기 위해 인간 검수 과정이 반드시 필요하다.

(2) Practical Limitations

본 시스템의 성능은 입력 강의자료의 품질에 크게 의존한다. 강의 노트가 불완전하거나 특정

주제가 충분히 설명되지 않은 경우 생성되는 문제 역시 해당 내용을 정확히 반영하지 못할 수 있다. 또한 현재 시스템은 인간 검수자가 직접 문제를 선택하고 수정 지시를 입력하는 Human-in-the-Loop 구조를 사용한다. 따라서 최종 시험지 생성 과정이 완전히 자동화되지는 않았으며, 여전히 일부 인적 개입이 요구된다.

마지막으로 본 시스템은 과학적 관리 과목을 대상으로 설계되었기 때문에 수학, 프로그래밍, 공학 설계 과목 등 다른 유형의 교과목에 적용할 경우 추가적인 프롬프트 수정과 시스템 조정이 필요할 수 있다.

5.2 Future Improvements

(1) Architectural Improvements

현재 시스템은 문제 생성과 검토를 하나의 LLM 기반 검토 단계에서 수행한다. 향후에는 Coverage Checker Agent, Difficulty Evaluation Agent, Duplicate Detection Agent 등을 독립적으로 구성하여 보다 세분화된 다중 에이전트 구조로 발전시킬 수 있다. 또한 현재는 인간 검수자가 직접 문제를 선택하지만, 향후에는 자동 점수화 및 랭킹 시스템을 도입하여 품질이 높은 문제를 우선 추천하는 기능을 추가할 수 있다.

(2) Prompt and Context Engineering

현재는 강의자료 전체를 통합하여 문제를 생성하고 있다. 강의 주제별로 내용을 분리한 뒤 각 주제의 중요도와 출제 빈도를 반영하는 프롬프트 전략 또한 적용할 수 있을 것으로 보인다. 또한 문제 유형별 전용 프롬프트를 설계하여 계산형, 응용형, 논술형 문제의 품질을 더욱 향상시킬 수 있을 것으로 기대된다.

(3) Integration with Additional Tools

Retrieval-Augmented Generation(RAG)을 도입하여 강의자료를 더욱 정확하게 참조하도록 개선할 수 있다. 이를 통해 Hallucination을 줄이고 강의 범위 기반 출제 정확도를 높일 수 있다. 또한 자동 채점 시스템과 연동하여 시험 생성뿐만 아니라 채점 기준 생성 및 답안 평가까지 수행하는 통합 시험 관리 시스템으로 확장할 수 있다.

5.3 Overall Evaluation

본 프로젝트는 강의자료와 출제 요구사항을 기반으로 시험 문제와 모범답안을 자동 생성하는 Agentic Work System을 설계 및 구현하였다. 시스템은 후보 문제 생성, 문제 검토, 인간 검수, 최종 시험지 생성의 과정을 통해 실제 시험에 활용 가능한 수준의 결과물을 생성하였다.

비록 LLM의 한계와 인간 검수 의존성 등의 제약이 존재하지만, 시험 출제 과정의 상당 부분을 자동화함으로써 교수자와 조교의 업무 부담을 크게 감소시킬 수 있음을 확인하였다. 따라서 본 시스템은 교육 분야에서 LLM 기반 Agentic Work System의 실질적인 활용 가능성을 보여주는 사례라고 평가할 수 있다.